

UNIVERSIDAD NACIONAL DEL ALTIPLANO
Escuela Profesional de Ingeniería de Sistemas

GUÍA DE PRÁCTICA DE LABORATORIO

PRÁCTICA N.º 09

Árboles Binarios de Búsqueda (BST)
Implementación en Python 3.11 y C++17

Curso:	Algoritmos y Estructuras de Datos — SIS210
Semana:	9
Lenguaje:	Python 3.11+ / C++17
Duración:	3 horas académicas
Docente Responsable:	Dr. Aldo Hernán Zanabria Gálvez
ORCID / RENACYT:	0000-0003-3314-8768 P0020608

Puno — Perú — 2025

I. OBJETIVO GENERAL

Comprender la estructura, propiedades y operaciones de los árboles binarios de búsqueda (BST) como estructura de datos jerárquica no lineal, implementando en Python 3.11 y C++17 un sistema de gestión de expedientes académicos de la Universidad Nacional del Altiplano de Puno (UNA-PUNO) que permita insertar, buscar, eliminar y recorrer registros en tiempo $O(\log n)$, comparando ambas implementaciones en rendimiento, memoria y facilidad de uso.

1.1 Objetivos Específicos

- Comprender la definición formal, propiedades y terminología de los árboles binarios y árboles binarios de búsqueda (BST).
- Implementar en Python 3.11 las operaciones básicas de un BST: inserción, búsqueda, eliminación y cuatro tipos de recorrido.
- Implementar en C++17 el mismo BST usando struct, punteros, unique_ptr, STL y manejo de excepciones.
- Analizar la complejidad temporal y espacial de cada operación usando notación Big-O.
- Aplicar el BST a un problema real: índice de expedientes académicos de estudiantes de la UNA-PUNO.
- Comparar el desempeño experimental de ambas implementaciones (Python vs C++) con distintos tamaños de dataset.

II. MARCO TEÓRICO

2.1 ¿Qué es un Árbol?

En Ciencias de la Computación, un árbol es una estructura de datos no lineal que organiza la información de forma jerárquica. Está compuesto por nodos conectados mediante aristas, con las siguientes características fundamentales (Cormen et al., 2022):

- Existe exactamente un nodo especial llamado raíz (root) que no tiene padre.
- Cada nodo (excepto la raíz) tiene exactamente un padre.
- No existen ciclos: la estructura es acíclica.
- Cada nodo puede tener cero o más hijos.

Los árboles son omnipresentes en sistemas reales: el sistema de archivos de Linux y Windows es un árbol de directorios; el DOM de HTML que los navegadores procesan es un árbol; las estructuras de datos internas de PostgreSQL para índices son árboles B+; y el scheduler del kernel Linux 6.x gestiona los procesos mediante un árbol Red-Black (una variante balanceada del BST).

2.2 Árbol Binario — Definición Formal

Un árbol binario es un árbol en el que cada nodo tiene a lo sumo dos hijos, denominados hijo izquierdo (left child) e hijo derecho (right child). Formalmente, un árbol binario T es:

$$T = \emptyset \text{ (árbol vacío)}$$

$$T = (\text{raíz}, T_{\text{izquierdo}}, T_{\text{derecho}}) \text{ donde } T_{\text{izq}} \text{ y } T_{\text{der}} \text{ son árboles binarios disjuntos}$$

Esta definición recursiva es la base de todos los algoritmos que operan sobre árboles binarios: cada operación se resuelve en la raíz y se delega recursivamente a los subárboles.

2.3 Terminología Esencial

La siguiente tabla resume los términos que se usan a lo largo de toda la práctica:

Término	Definición	Ejemplo / Nota
Raíz (Root)	Nodo sin padre. Punto de entrada al árbol.	$depth(raíz) = 0$
Hoja (Leaf)	Nodo sin hijos (izquierdo = derecho = None).	También llamado nodo terminal
Altura (Height)	Longitud del camino más largo desde la raíz hasta una hoja.	$h = O(\log n)$ si balanceado
Profundidad	Longitud del camino desde la raíz hasta el nodo n.	$depth(raíz) = 0$
Nodo interno	Nodo que tiene al menos un hijo.	Raíz siempre es interno (si $n > 1$)
Subárbol	Árbol formado por un nodo y todos sus descendientes.	Cada nodo es raíz de su propio subárbol
Árbol perfecto	Todos los niveles completos. $n = 2^{(h+1)} - 1$ nodos.	7 nodos $\rightarrow h = 2$
Árbol completo	Todos los niveles completos excepto posiblemente el último, con nodos a la izquierda.	Base de los heaps binarios

2.4 Árbol Binario de Búsqueda (BST)

El árbol binario de búsqueda (BST, Binary Search Tree) es un árbol binario con la siguiente invariante de orden, que debe cumplirse para todo nodo n del árbol:

Para todo nodo n: $max(subárbol_izquierdo) < n.clave < min(subárbol_derecho)$

Esta propiedad se mantiene recursivamente en todos los subárboles, no solo en la raíz. Gracias a ella, la búsqueda en un BST funciona como la búsqueda binaria: en cada nodo se descarta la mitad del árbol, logrando $O(\log n)$ en el caso promedio.

Ejemplo con las claves 50, 30, 70, 20, 40, 60, 80 insertadas en ese orden:

Inserción: 50 → 30 → 70 → 20 → 40 → 60 → 80

Árbol resultante:

50

/ \

30 70

/ \ / \

20 40 60 80

← raíz, altura = 0

← nivel 1

← nivel 2, hojas

Verificación de la invariante BST:

Nodo 50: izquierdo={20,30,40} todos < 50 ✓

derecho= {60,70,80} todos > 50 ✓

Nodo 30: izquierdo={20} < 30 ✓ derecho={40} > 30 ✓

Nodo 70: izquierdo={60} < 70 ✓ derecho={80} > 70 ✓

```
In-order (izq→raíz→der): 20, 30, 40, 50, 60, 70, 80 ← ¡ordenado! ✓
```

2.5 Operaciones del BST

Las tres operaciones fundamentales del BST aprovechan la invariante de orden para navegar eficientemente:

Operación	Algoritmo	Complejidad	Peor caso
Insertar	Si árbol vacío: crear nodo. Si clave < raíz: ir a izquierda. Si clave > raíz: ir a derecha. Repetir hasta encontrar posición vacía.	$O(\log n)$	$O(n)$
Buscar	Comparar con raíz. Si igual: encontrado. Si clave < raíz: buscar en izquierda. Si clave > raíz: buscar en derecha.	$O(\log n)$	$O(n)$
Eliminar	3 casos: (1) nodo hoja → eliminar directo; (2) un hijo → reemplazar con el hijo; (3) dos hijos → reemplazar con sucesor in-order (mínimo del subárbol derecho).	$O(\log n)$	$O(n)$

**Caso degenerado:** Si se insertan las claves en orden creciente (ej. 10, 20, 30, 40...), el BST se convierte en una lista enlazada con altura $h = n - 1$, haciendo que buscar, insertar y eliminar sean $O(n)$. Esta limitación motiva los árboles AVL (semana 11) y Red-Black Trees (semana 12).

2.6 Recorridos del Árbol Binario

Un recorrido (traversal) visita exactamente una vez cada nodo del árbol. Existen cuatro estrategias clásicas, cada una con aplicaciones distintas (Sedgewick & Wayne, 2011):

Recorrido	Orden de visita	Aplicación práctica
In-Order	Izquierdo → Raíz → Derecho	Produce lista ordenada ascendente. Usado para listar registros ordenados por clave.
Pre-Order	Raíz → Izquierdo → Derecho	Serializar/copiar el árbol. Construir expresiones prefijas.
Post-Order	Izquierdo → Derecho → Raíz	Liberar memoria del árbol. Evaluar expresiones postfijas.
BFS (por niveles)	Nivel a nivel, izquierda → derecha (usa cola)	Encontrar el camino más corto. Visualizar el árbol nivel por nivel.

Ejemplo con el árbol del punto 2.4 (claves 50, 30, 70, 20, 40, 60, 80):

In-Order	→ [20, 30, 40, 50, 60, 70, 80]	← lista ordenada ascendente
Pre-Order	→ [50, 30, 20, 40, 70, 60, 80]	← raíz primero
Post-Order	→ [20, 40, 30, 60, 80, 70, 50]	← raíz al final
BFS	→ [50, 30, 70, 20, 40, 60, 80]	← nivel por nivel

2.7 Complejidad Algorítmica — Notación Big-O

La notación Big-O describe el comportamiento asintótico del tiempo de ejecución y el uso de memoria en función del tamaño n del dataset (Cormen et al., 2022):

Operación	Mejor caso	Caso promedio	Peor caso	Espacio extra
Insertar	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$ stack
Buscar	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$ stack
Eliminar	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$ stack
Recorridos (in/pre/post/BFS)	$O(n)$	$O(n)$	$O(n)$	$O(n)$ lista
Espacio total árbol	$O(n)$	$O(n)$	$O(n)$	—

2.8 El BST en la Industria y la Ciencia

Los árboles binarios de búsqueda y sus variantes balanceadas son la estructura de datos subyacente en los sistemas más críticos del mundo:

- Linux Kernel 6.x: usa Red-Black Trees (BST autobalanceado) para el scheduler de procesos CFS y para gestionar las áreas de memoria virtual (VMAs) de cada proceso. Cada llamada a `mmap()` realiza una búsqueda $O(\log n)$ en este árbol.
- PostgreSQL 16 / Oracle DB: el tipo de índice por defecto (btree) para cualquier columna indexada es un árbol B+-Tree, variante del BST optimizada para almacenamiento en disco.
- Microsoft .NET: `SortedDictionary<K,V>` y `SortedSet<T>` de la biblioteca estándar están implementados internamente como Red-Black Trees.
- scikit-learn (Python): `DecisionTreeClassifier` implementa árboles de decisión binarios donde cada nodo interno es un umbral de partición del espacio de características, aplicados en diagnóstico médico, clasificación de especies y detección de fraudes.
- GeoAI / INEI Puno: KD-Tree (extensión multidimensional del BST) para indexar coordenadas GPS de los hogares en los 109 distritos de la Región Puno, permitiendo consultas espaciales en $O(\log n)$ para análisis de pobreza multidimensional.

III. MATERIALES Y HERRAMIENTAS

Herramienta	Uso
Python 3.11+	Lenguaje principal de implementación
Google Colab / VS Code	Entorno de ejecución (Colab no requiere instalación)
dataclasses (stdlib)	Definición limpia de la clase Estudiante con @dataclass
typing (stdlib)	Anotaciones de tipo: Optional, List para mayor legibilidad
collections.deque (stdlib)	Cola eficiente O(1) para implementar el recorrido BFS
time (stdlib)	Medición de tiempos de ejecución para benchmarks
random (stdlib)	Generación de datos sintéticos para los experimentos

IV. INSTRUCCIONES PREVIAS

- Abrir Google Colab en colab.research.google.com y crear un nuevo notebook con el nombre: Practica_09_BST_ApellidoNombre.
- Cada actividad se implementa en una celda de código separada. Ejecutar con Shift+Enter antes de pasar a la siguiente.
- Leer completamente la explicación de cada actividad antes de escribir el código.
- El código proporcionado es la estructura base. El estudiante debe completar los métodos marcados con # TODO.
- Al finalizar, descargar el notebook (.ipynb) y adjuntarlo junto con un breve informe en el aula virtual.

V. DESARROLLO DE ACTIVIDADES

ACTIVIDAD 1: Definir la estructura de datos del Nodo y del Estudiante

Objetivo: Crear las clases base que representan un expediente académico y un nodo del BST usando @dataclass de Python.

Antes de construir el árbol necesitamos definir qué información guarda cada nodo. Usamos @dataclass para aprovechar la generación automática de __init__, __repr__ y __eq__, y la clase Enum para el estado académico:

```
# — Celda 1: Estructura de datos —————
from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum, auto
from typing import Optional, List
from collections import deque

# Estado académico del estudiante
class EstadoAcademico(Enum):
    ACTIVO = auto()
    EGRESADO = auto()
```

```

RETIRADO = auto()
SUSPENDIDO = auto()

# Datos del expediente (Value Object)
@dataclass
class Estudiante:
    """Expediente académico – clave del BST: atributo codigo."""
    codigo: int # clave primaria del BST [8 dígitos]
    nombre: str
    escuela: str
    ppa: float # Promedio Ponderado Acumulado [0.0, 20.0]
    creditos: int
    estado: EstadoAcademico
    semestre_ingreso: str

    def __post_init__(self):
        # Validación de datos al crear el objeto
        if not (10_000_000 <= self.codigo <= 29_999_999):
            raise ValueError(f"Código inválido: {self.codigo}")
        if not (0.0 <= self.ppa <= 20.0):
            raise ValueError(f"PPA fuera de rango [0,20]: {self.ppa}")

# Nodo interno del BST
@dataclass
class NodoBST:
    """Cada nodo guarda un Estudiante y dos referencias al hijo izq/der."""
    dato: Estudiante
    izquierdo: Optional[NodoBST] = field(default=None, repr=False)
    derecho: Optional[NodoBST] = field(default=None, repr=False)

# — Prueba rápida —
e1 = Estudiante(20210500, 'Mamani Quispe, Juan', 'Ing. Sistemas',
                15.8, 120, EstadoAcademico.ACTIVO, '2021-I')
nodo = NodoBST(dato=e1)
print('Nodo creado:', nodo.dato.codigo, nodo.dato.nombre)
print('Hijo izquierdo:', nodo.izquierdo) # None
print('Hijo derecho: ', nodo.derecho)    # None

```

Observe que `NodoBST` tiene tres campos: `dato` (el expediente), `izquierdo` y `derecho` (referencias al siguiente nodo). Al inicio, ambos hijos son `None`, es decir, el nodo es una hoja del árbol.

ACTIVIDAD 2: Implementar la clase `ArbolAcademico`: insertar y recorrer

Objetivo: Construir el árbol BST con la operación de inserción recursiva y los cuatro recorridos.

La clase `ArbolAcademico` encapsula toda la lógica del árbol. El método `insertar()` navega el árbol comparando códigos: si el nuevo código es menor, va a la izquierda; si es mayor, va a la derecha; si llega a `None`, crea el nodo. Los recorridos `in_order()`, `pre_order()`, `post_order()` y `bfs()` visitan cada nodo exactamente una vez:

```

# — Celda 2: Clase ArbolAcademico – insertar y recorridos —
class ArbolAcademico:
    """BST para gestión de expedientes académicos UNA-PUNO."""

    def __init__(self) -> None:
        self._raiz: Optional[NodoBST] = None

# — INSERTAR —

```

```

def insertar(self, e: Estudiante) -> None:
    """Inserta e en el árbol. Lanza ValueError si el código ya existe."""
    self._raiz = self._insertar(self._raiz, e)

def _insertar(self, nodo: Optional[NodoBST],
              e: Estudiante) -> NodoBST:
    if nodo is None:
        return NodoBST(dato=e)
    if e.codigo < nodo.dato.codigo:
        nodo.izquierdo = self._insertar(nodo.izquierdo, e)
    elif e.codigo > nodo.dato.codigo:
        nodo.derecho = self._insertar(nodo.derecho, e)
    else:
        raise ValueError(f"Código duplicado: {e.codigo}")
    return nodo

# — IN-ORDER: izquierdo → raíz → derecho —————
def in_order(self) -> List[Estudiante]:
    """Retorna lista ordenada ascendente por código."""
    resultado: List[Estudiante] = []
    self._in_order(self._raiz, resultado)
    return resultado

def _in_order(self, nodo, res):
    if nodo is None: return
    self._in_order(nodo.izquierdo, res) # 1. visitar izquierda
    res.append(nodo.dato)                # 2. visitar raíz
    self._in_order(nodo.derecho, res)    # 3. visitar derecha

# — PRE-ORDER: raíz → izquierdo → derecho —————
def pre_order(self) -> List[Estudiante]:
    resultado: List[Estudiante] = []
    self._pre_order(self._raiz, resultado)
    return resultado

def _pre_order(self, nodo, res):
    if nodo is None: return
    res.append(nodo.dato)                # 1. visitar raíz primero
    self._pre_order(nodo.izquierdo, res)
    self._pre_order(nodo.derecho, res)

# — BFS: recorrido por niveles (usa collections.deque) —————
def bfs(self) -> List[Estudiante]:
    """Level-order: devuelve nodos nivel a nivel, izquierda-derecha."""
    if self._raiz is None: return []
    resultado, cola = [], deque([self._raiz])
    while cola:
        nodo = cola.popleft()            # O(1) con deque
        resultado.append(nodo.dato)
        if nodo.izquierdo: cola.append(nodo.izquierdo)
        if nodo.derecho: cola.append(nodo.derecho)
    return resultado

def altura(self) -> int:
    """Altura del árbol (-1 si está vacío)."""
    return self._altura(self._raiz)

def _altura(self, nodo) -> int:
    if nodo is None: return -1
    return 1 + max(self._altura(nodo.izquierdo),
                  self._altura(nodo.derecho))

# — Prueba: insertar 7 estudiantes y recorrer —————
arbol = ArbolAcademico()
datos = [

```



```

    Estudiante(20210500, 'Mamani Quispe, Juan', 'Ing. Sistemas', 15.8, 120,
EstadoAcademico.ACTIVO, '2021-I'),
    Estudiante(20210300, 'Huanca Apaza, Maria', 'Ing. Civil', 14.2, 110,
EstadoAcademico.ACTIVO, '2021-I'),
    Estudiante(20210700, 'Condori Flores, Pedro', 'Medicina', 17.1, 130,
EstadoAcademico.ACTIVO, '2021-I'),
    Estudiante(20210100, 'Ticona Lupaca, Rosa', 'Contabilidad', 12.0, 90,
EstadoAcademico.SUSPENDIDO, '2021-I'),
    Estudiante(20210400, 'Larico Ccama, Carlos', 'Ing. Sistemas', 16.5, 115,
EstadoAcademico.ACTIVO, '2021-I'),
    Estudiante(20210600, 'Cutipa Vargas, Elena', 'Agronomia', 13.7, 100,
EstadoAcademico.ACTIVO, '2021-I'),
    Estudiante(20210900, 'Pari Choque, Luis', 'Ing. Sistemas', 18.3, 140,
EstadoAcademico.EGRESADO, '2021-I'),
]
for e in datos: arbol.insertar(e)

print(f'Altura del árbol: {arbol.altura()}')
print()
print('In-Order (ordenado por código):')
for e in arbol.in_order():
    print(f' {e.codigo} {e.nombre:<35} PPA: {e.ppa}')
print()
print('BFS (nivel por nivel):')
print([e.codigo for e in arbol.bfs()])

```

Salida esperada del In-Order: los códigos deben aparecer en orden creciente (20210100, 20210300, ..., 20210900), lo que confirma que la invariante BST se mantiene correctamente. El BFS mostrará primero la raíz (20210500), luego el nivel 1, etc.

ACTIVIDAD 3: Implementar búsqueda y eliminación

Objetivo: Agregar los métodos `buscar()` y `eliminar()` a la clase `ArbolAcademico`, manejando los tres casos de eliminación.

La búsqueda navega el árbol comparando la clave buscada con el nodo actual: si es menor va a la izquierda, si es mayor a la derecha, si es igual retorna el dato. La eliminación es la operación más compleja: debe mantener la invariante BST después de retirar el nodo. El Caso 3 (dos hijos) usa el sucesor in-order para no romper el orden:

```

# — Celda 3: Agregar búsqueda y eliminación a ArbolAcademico —————
# (pegue este código debajo del __init__ dentro de la clase)

# — BUSCAR —————
def buscar(self, codigo: int) -> Optional[Estudiante]:
    """Retorna el Estudiante con ese código, o None si no existe."""
    nodo = self._buscar(self._raiz, codigo)
    return nodo.dato if nodo else None

def _buscar(self, nodo, codigo):
    if nodo is None or nodo.dato.codigo == codigo:
        return nodo # encontrado o no existe
    if codigo < nodo.dato.codigo:
        return self._buscar(nodo.izquierdo, codigo) # ir izquierda
    return self._buscar(nodo.derecho, codigo) # ir derecha

# — ELIMINAR —————
def eliminar(self, codigo: int) -> None:

```

```

        """Elimina el nodo con ese código. Lanza KeyError si no existe."""
        if self.buscar(codigo) is None:
            raise KeyError(f"Código no encontrado: {codigo}")
        self._raiz = self._eliminar(self._raiz, codigo)

    def _eliminar(self, nodo, codigo):
        if nodo is None:
            return None

        if codigo < nodo.dato.codigo:                # ir izquierda
            nodo.izquierdo = self._eliminar(nodo.izquierdo, codigo)
        elif codigo > nodo.dato.codigo:                # ir derecha
            nodo.derecho = self._eliminar(nodo.derecho, codigo)
        else:
            # — NODO ENCONTRADO: tres casos —————
            if nodo.izquierdo is None:                # Caso 1 y 2: sin hijo izq
                return nodo.derecho
            if nodo.derecho is None:                    # Caso 2: sin hijo der
                return nodo.izquierdo
            # Caso 3: dos hijos → buscar sucesor in-order
            sucesor = self._minimo(nodo.derecho)      # mínimo subárbol der
            nodo.dato = sucesor.dato                  # copiar datos
            nodo.derecho = self._eliminar(nodo.derecho, sucesor.dato.codigo)

        return nodo

    def _minimo(self, nodo):    # el nodo más a la izquierda
        while nodo.izquierdo:
            nodo = nodo.izquierdo
        return nodo

# — Pruebas de búsqueda y eliminación —————
encontrado = arbol.buscar(20210700)
if encontrado:
    print(f'Encontrado: {encontrado.codigo} {encontrado.nombre} PPA: {encontrado.ppa}')

no_existe = arbol.buscar(99999999)
print(f'Buscar 99999999: {no_existe}')    # None

print(f'Nodos antes de eliminar: {len(arbol.in_order())}')
arbol.eliminar(20210300)    # eliminar nodo con dos hijos
print(f'Nodos después de eliminar 20210300: {len(arbol.in_order())}')
print('In-Order después:', [e.codigo for e in arbol.in_order()])

```

Observe que al eliminar el nodo 20210300 (que tiene dos hijos: 20210100 y 20210400), su sucesor in-order es 20210400 (el mínimo del subárbol derecho). El código 20210400 ocupa el lugar de 20210300 y la invariante BST se mantiene.

ACTIVIDAD 4: Agregar consultas y estadísticas

Objetivo: Implementar consultas por rango de PPA y estadísticas generales del árbol.

Con el árbol construido podemos hacer consultas más ricas que una simple búsqueda por clave. La consulta por rango de PPA recorre todo el árbol in-order (ya ordenado por código) y filtra los registros que cumplen la condición:

```
# — Celda 4: Consultas y estadísticas —————
```

```

import statistics

# — CONSULTAS —
def por_rango_ppa(self, ppa_min: float,
                    ppa_max: float = 20.0) -> List[Estudiante]:
    """Estudiantes con PPA en [ppa_min, ppa_max]."""
    return [e for e in self.in_order()
            if ppa_min <= e.ppa <= ppa_max]

def por_escuela(self, escuela: str) -> List[Estudiante]:
    return [e for e in self.in_order() if e.escuela == escuela]

def por_estado(self, estado: EstadoAcademico) -> List[Estudiante]:
    return [e for e in self.in_order() if e.estado == estado]

# — ESTADÍSTICAS —
def estadisticas(self) -> dict:
    """Calcula estadísticas del árbol en O(n)."""
    todos = self.in_order()
    if not todos:
        return {}
    ppas = [e.ppa for e in todos]
    return {
        'total_nodos' : len(todos),
        'altura'      : self.altura(),
        'ppa_promedio' : round(statistics.mean(ppas), 2),
        'ppa_minimo'   : min(ppas),
        'ppa_maximo'    : max(ppas),
        'total_activos' : sum(1 for e in todos
                             if e.estado == EstadoAcademico.ACTIVO),
    }

# — Pruebas —
print('Estudiantes con PPA >= 15.0:')
for e in arbol.por_rango_ppa(15.0):
    print(f' {e.codigo} {e.nombre:<35} PPA: {e.ppa}')

print()
print('Estadísticas del árbol:')
for k, v in arbol.estadisticas().items():
    print(f' {k:<20}: {v}')

```

ACTIVIDAD 5: Visualizar la estructura del árbol

Objetivo: Imprimir el árbol en la consola usando representación ASCII para verificar visualmente la estructura BST.

Una herramienta invaluable para depurar un BST es poder ver su estructura. La siguiente función imprime el árbol rotado 90° hacia la izquierda: la raíz aparece al extremo izquierdo y las hojas al extremo derecho:

```

# — Celda 5: Visualización ASCII del árbol —

def imprimir_arbol(self) -> None:
    """Imprime el árbol en ASCII rotado 90°."""
    print('\n— Estructura del BST —')
    self._imprimir(self._raiz, '', False)

def _imprimir(self, nodo, prefix: str, is_left: bool) -> None:

```

```

        if nodo is None: return
        conector = '├─ ' if is_left else '└─ '
        print(f'{prefix}{conector}{nodo.dato.codigo} [PPA:{nodo.dato.ppa:.1f}]')
        ext = '│ ' if is_left else ' '
        self._imprimir(nodo.izquierdo, prefix + ext, True)
        self._imprimir(nodo.derecho, prefix + ext, False)

# — Ver el árbol —————
arbol.imprimir_arbol()

# Salida esperada (árbol con 6 nodos tras eliminar 20210300):
#
# — Estructura del BST —
# └─ 20210500 [PPA:15.8]
#   └─ 20210400 [PPA:16.5] ← ocupa lugar de 20210300 (sucesor)
#       └─ 20210100 [PPA:12.0]
#           └─ None
#   └─ 20210700 [PPA:17.1]
#       └─ 20210600 [PPA:13.7]
#           └─ 20210900 [PPA:18.3]

```

ACTIVIDAD 6: Benchmark experimental — BST vs lista vs diccionario

Objetivo: Medir y comparar los tiempos de inserción y búsqueda del BST, una lista ordenada y un diccionario Python con 100, 1 000, 10 000 y 100 000 registros.

El objetivo de este experimento es verificar empíricamente las diferencias de complejidad. El BST debería mostrar un crecimiento $O(\log n)$, el diccionario $O(1)$ amortizado (tabla hash) y la lista $O(n)$ en búsqueda lineal. Use datos con códigos generados aleatoriamente para evitar el árbol degenerado:

```

# — Celda 6: Benchmark experimental —————
import time, random

def generar_datos(n: int) -> list:
    codigos = random.sample(range(20_000_000, 29_999_999), n)
    return [Estudiante(cod, f'Estudiante_{i}', 'Ingenieria',
                        round(random.uniform(8.0, 20.0), 1), 100,
                        EstadoAcademico.ACTIVO, '2024-I')
            for i, cod in enumerate(codigos)]

def benchmark(n: int):
    datos = generar_datos(n)
    buscar_cod = datos[n // 2].codigo # código a buscar

    # — BST —————
    arbol_b = ArbolAcademico()
    t0 = time.perf_counter()
    for e in datos: arbol_b.insertar(e)
    t_ins_bst = (time.perf_counter() - t0) * 1000

    t0 = time.perf_counter()
    arbol_b.buscar(buscar_cod)
    t_bus_bst = (time.perf_counter() - t0) * 1000

    # — Diccionario Python (hash map) —————
    diccionario = {}
    t0 = time.perf_counter()
    for e in datos: diccionario[e.codigo] = e

```

```

t_ins_dic = (time.perf_counter() - t0) * 1000

t0 = time.perf_counter()
_ = diccionario.get(buscar_cod)
t_bus_dic = (time.perf_counter() - t0) * 1000

print(f'N={n:>8} | BST ins:{t_ins_bst:7.2f}ms bus:{t_bus_bst:.4f}ms |
      Dict ins:{t_ins_dic:7.2f}ms bus:{t_bus_dic:.4f}ms |
      Altura BST: {arbol_b.altura()}')

print('N          | BST inserción   búsqueda | Dict inserción   búsqueda |
      Altura')
print('-' * 85)
for n in [100, 1_000, 10_000, 100_000]:
    benchmark(n)

```

Complete la siguiente tabla con los resultados obtenidos en su equipo y analice el comportamiento. El campo 'Altura BST' le permite verificar si el árbol se acerca a $\log_2(N)$:

N	BST ins. (ms)	BST búsq. (ms)	Dict ins. (ms)	Dict búsq. (ms)	Altura
100					
1 000					
10 000					
100 000					

ACTIVIDAD 7: Experimentación del estudiante

Objetivo: Modificar el código y analizar el comportamiento del BST ante distintos escenarios.

Realice las siguientes modificaciones de forma independiente y anote sus observaciones:

1. Árbol degenerado: inserte las claves en orden creciente en lugar de aleatorio (codigos = list(range(20_000_000, 20_000_000+n))). Compare la altura resultante con el caso aleatorio. ¿A qué se parece visualmente el árbol?
2. Agregar post_order: implemente el método post_order() siguiendo el patrón de in_order() pero visitando izquierdo → derecho → raíz. Verifique con las claves del ejemplo del marco teórico.
3. Buscar el máximo: implemente el método maximo() que retorne el Estudiante con el código más alto. ¿En qué dirección del árbol siempre se encuentra? ¿Cuál es su complejidad?
4. Rango de código: implemente buscar_rango_codigo(cod_min, cod_max) que use la propiedad BST para descartar subárboles completos, logrando mejor rendimiento que filtrar toda la lista in_order().

PARTE 2

Implementación en C++17

struct · unique_ptr · STL · OOP · Manejo de excepciones

En esta parte replicará exactamente la misma funcionalidad desarrollada en Python, pero usando C++17. Observará las diferencias en sintaxis, gestión de memoria y rendimiento. Necesitará un compilador compatible: g++ 10+ (Linux/macOS) o MinGW-w64 (Windows).

Comando de compilación para todas las actividades de C++:

```
g++ -std=c++17 -O2 -Wall -o bst_academico ArbolAcademico.cpp && ./bst_academico
```

ACTIVIDAD 8: C++ — Definir struct Estudiante y NodoBST

Objetivo: Crear las estructuras de datos base en C++17 equivalentes a los @dataclass de Python, usando enum class y unique_ptr para gestión automática de memoria.

En C++ no existe @dataclass, pero struct cumple la misma función. Usamos enum class (C++11) para el estado académico y std::unique_ptr (C++14) en lugar de referencias Python. El unique_ptr garantiza que la memoria se libera automáticamente cuando el nodo se destruye, sin necesidad de llamar a delete manualmente — este patrón se llama RAII (Resource Acquisition Is Initialization):

```
// — ArbolAcademico.hpp
#pragma once
#include <string>
#include <vector>
#include <queue>
#include <memory>           // unique_ptr
#include <optional>         // optional<T> (C++17)
#include <stdexcept>        // runtime_error, invalid_argument
#include <iostream>
#include <iomanip>

namespace una_puno {

// Estado académico – equivalente al Enum de Python
enum class EstadoAcademico {
    ACTIVO, EGRESADO, RETIRADO, SUSPENDIDO
};

// Convierte enum a string para imprimir
inline std::string estadoStr(EstadoAcademico e) {
    switch(e) {
        case EstadoAcademico::ACTIVO:      return "ACTIVO";
        case EstadoAcademico::EGRESADO:    return "EGRESADO";
        case EstadoAcademico::RETIRADO:    return "RETIRADO";
        case EstadoAcademico::SUSPENDIDO:  return "SUSPENDIDO";
        default: return "DESCONOCIDO";
    }
}

// Datos del expediente – equivalente a @dataclass Estudiante
struct Estudiante {
    int          codigo;           // clave primaria del BST
    std::string  nombre;
};
```

```

std::string    escuela;
float          ppa;           // [0.0, 20.0]
int            credits;
EstadoAcademico estado;
std::string    semestre_ingreso;

// Constructor con validación (equivale a __post_init__ de Python)
Estudiante(int cod, std::string nom, std::string esc,
            float pp, int cred, EstadoAcademico est, std::string sem)
    : codigo(cod), nombre(std::move(nom)), escuela(std::move(esc)),
      ppa(pp), credits(cred), estado(est),
      semestre_ingreso(std::move(sem))
{
    if (cod < 10000000 || cod > 29999999)
        throw std::invalid_argument("Codigo invalido: " +
std::to_string(cod));
    if (pp < 0.0f || pp > 20.0f)
        throw std::invalid_argument("PPA fuera de rango [0,20]");
}

void print() const {
    std::cout << std::left
                << std::setw(10) << codigo
                << std::setw(35) << nombre
                << std::setw(20) << escuela
                << "PPA:" << std::fixed << std::setprecision(1) << ppa
                << " " << estadoStr(estado) << '\n';
}
};

// Nodo del BST – usa unique_ptr para gestión automática de memoria (RAII)
struct NodoBST {
    Estudiante          dato;
    std::unique_ptr<NodoBST> izquierdo;  // NULL si no hay hijo
    std::unique_ptr<NodoBST> derecho;

    explicit NodoBST(Estudiante e)
        : dato(std::move(e)),
          izquierdo(nullptr), derecho(nullptr) {}
};

} // namespace una_puno

```

La diferencia clave con Python: en Python el recolector de basura libera la memoria automáticamente. En C++, `unique_ptr` cumple ese rol — cuando el nodo padre se destruye, sus hijos `unique_ptr` también se destruyen en cadena, liberando todo el árbol sin un solo `delete` explícito.

ACTIVIDAD 9: C++ — Implementar insertar y recorridos

Objetivo: Construir la clase `ArbolAcademico` en C++ con inserción recursiva y los cuatro recorridos, equivalentes a los métodos Python de la Actividad 2.

La lógica recursiva es idéntica a Python. La diferencia está en la sintaxis: usamos `std::unique_ptr<NodoBST>&` (referencia a puntero) en insertar para poder modificar el árbol in-place, `std::queue` de la STL para BFS, y `std::vector` para acumular resultados:

```
// — ArbolAcademico.cpp (parte 1: insertar y recorridos) —————
```

```

#include "ArbolAcademico.hpp"

namespace una_puno {

class ArbolAcademico {
public:
    ArbolAcademico() = default;    // constructor por defecto

    // — INSERTAR —
    void insertar(Estudiante e) {
        insertar_(raiz, std::move(e));
    }

private:
    std::unique_ptr<NodoBST> raiz;    // raíz del árbol

    // Versión recursiva interna (recibe referencia al unique_ptr)
    void insertar_(std::unique_ptr<NodoBST>& nodo, Estudiante e) {
        if (!nodo) {                // posición vacía encontrada
            nodo = std::make_unique<NodoBST>(std::move(e));
            return;
        }
        if (e.codigo < nodo->dato.codigo)
            insertar_(nodo->izquierdo, std::move(e));
        else if (e.codigo > nodo->dato.codigo)
            insertar_(nodo->derecho, std::move(e));
        else
            throw std::runtime_error("Codigo duplicado: " +
std::to_string(e.codigo));
    }

public:
    // — IN-ORDER: izquierdo → raíz → derecho —
    std::vector<Estudiante> inOrder() const {
        std::vector<Estudiante> resultado;
        inOrder_(raiz.get(), resultado);
        return resultado;
    }

private:
    void inOrder_(const NodoBST* n, std::vector<Estudiante>& r) const {
        if (!n) return;
        inOrder_(n->izquierdo.get(), r);    // 1. visitar izquierda
        r.push_back(n->dato);                // 2. visitar raíz
        inOrder_(n->derecho.get(), r);        // 3. visitar derecha
    }

public:
    // — PRE-ORDER: raíz → izquierdo → derecho —
    std::vector<Estudiante> preOrder() const {
        std::vector<Estudiante> resultado;
        preOrder_(raiz.get(), resultado);
        return resultado;
    }

private:
    void preOrder_(const NodoBST* n, std::vector<Estudiante>& r) const {
        if (!n) return;
        r.push_back(n->dato);                // 1. raíz primero
        preOrder_(n->izquierdo.get(), r);
        preOrder_(n->derecho.get(), r);
    }

public:
    // — BFS: recorrido por niveles con std::queue —

```



```

std::vector<Estudiante> bfs() const {
    std::vector<Estudiante> resultado;
    if (!raiz) return resultado;
    std::queue<const NodoBST*> cola;
    cola.push(raiz.get());
    while (!cola.empty()) {
        const NodoBST* curr = cola.front(); cola.pop();
        resultado.push_back(curr->dato);
        if (curr->izquierdo) cola.push(curr->izquierdo.get());
        if (curr->derecho) cola.push(curr->derecho.get());
    }
    return resultado;
}

int altura() const { return altura_(raiz.get()); }
bool estaVacio() const { return raiz == nullptr; }

private:
    int altura_(const NodoBST* n) const {
        if (!n) return -1;
        return 1 + std::max(altura_(n->izquierdo.get()),
                           altura_(n->derecho.get()));
    }
};

} // namespace una_puno

```

Note la diferencia con Python: en C++ usamos `raiz.get()` para obtener un puntero crudo (raw pointer) solo para lectura dentro de los métodos recursivos privados. Nunca se usa `delete` — el `unique_ptr` gestiona la vida del objeto.

ACTIVIDAD 10: C++ — Implementar buscar y eliminar

Objetivo: Agregar los métodos `buscar()` usando `std::optional` y `eliminar()` con los tres casos, equivalentes a la Actividad 3 de Python.

`std::optional<T>` (C++17) es el equivalente directo de `Optional[T]` en Python: permite retornar un valor o "nada" sin usar `nullptr` ni excepciones para el caso no-encontrado. La eliminación usa `std::move` para transferir ownership del `unique_ptr` al reemplazar nodos:

```

// — ArbolAcademico.cpp (parte 2: buscar y eliminar) —————

// — BUSCAR —————
std::optional<Estudiante> buscar(int codigo) const {
    const NodoBST* nodo = buscar_(raiz.get(), codigo);
    if (nodo) return nodo->dato;
    return std::nullopt;    // equivalente a return None en Python
}

private:
    const NodoBST* buscar_(const NodoBST* n, int cod) const {
        if (!n || n->dato.codigo == cod) return n;
        if (cod < n->dato.codigo)
            return buscar_(n->izquierdo.get(), cod);
        return buscar_(n->derecho.get(), cod);
    }

public:

```

```

// — ELIMINAR —
void eliminar(int codigo) {
    if (!buscar(codigo))
        throw std::runtime_error("Codigo no encontrado: " +
std::to_string(codigo));
    raiz = eliminar_(std::move(raiz), codigo);
}

private:
std::unique_ptr<NodoBST> eliminar_(
    std::unique_ptr<NodoBST> nodo, int cod) {
    if (!nodo) return nullptr;

    if (cod < nodo->dato.codigo)
        nodo->izquierdo = eliminar_(std::move(nodo->izquierdo), cod);
    else if (cod > nodo->dato.codigo)
        nodo->derecho = eliminar_(std::move(nodo->derecho), cod);
    else {
        // — Tres casos —
        if (!nodo->izquierdo) // Caso 1/2: sin hijo izquierdo
            return std::move(nodo->derecho);
        if (!nodo->derecho) // Caso 2: sin hijo derecho
            return std::move(nodo->izquierdo);
        // Caso 3: dos hijos → buscar sucesor in-order (mínimo der.)
        NodoBST* suc = minimo_(nodo->derecho.get());
        nodo->dato = suc->dato; // copiar datos del sucesor
        nodo->derecho = eliminar_(std::move(nodo->derecho),
                                suc->dato.codigo);
    }
    return nodo;
}

NodoBST* minimo_(NodoBST* n) const {
    while (n->izquierdo) n = n->izquierdo.get();
    return n;
}

```

ACTIVIDAD 11: C++ — Consultas, estadísticas e impresión ASCII

Objetivo: Completar la clase con las consultas por rango de PPA, estadísticas y la visualización ASCII del árbol equivalentes a las Actividades 4 y 5 de Python.

```

// — ArbolAcademico.cpp (parte 3: consultas, estadísticas, impresión) —

// — CONSULTA POR RANGO DE PPA —
std::vector<Estudiante> porRangoPPA(float ppa_min,
                                float ppa_max = 20.0f) const {
    auto todos = inOrder(); // ya en orden por código
    std::vector<Estudiante> resultado;
    std::copy_if(todos.begin(), todos.end(),
                std::back_inserter(resultado),
                [ppa_min, ppa_max](const Estudiante& e){
                    return e.ppa >= ppa_min && e.ppa <= ppa_max;
                });
    return resultado;
}

// — ESTADÍSTICAS —
void estadisticas() const {
    auto todos = inOrder();
    if (todos.empty()) { std::cout << "Arbol vacio\n"; return; }
}

```

```

float suma = 0.0f, mn = 20.0f, mx = 0.0f;
int  activos = 0;
for (const auto& e : todos) {
    suma += e.ppa;
    if (e.ppa < mn) mn = e.ppa;
    if (e.ppa > mx) mx = e.ppa;
    if (e.estado == EstadoAcademico::ACTIVO) activos++;
}
std::cout << std::fixed << std::setprecision(2)
    << "  Total nodos   : " << todos.size()    << '\n'
    << "  Altura       : " << altura()          << '\n'
    << "  PPA promedio  : " << suma/todos.size() << '\n'
    << "  PPA minimo    : " << mn                << '\n'
    << "  PPA maximo    : " << mx                << '\n'
    << "  Activos      : " << activos            << '\n';
}

// — IMPRESIÓN ASCII del árbol —————
void imprimirArbol() const {
    std::cout << "\n-- Estructura del BST --\n";
    imprimir_(raiz.get(), "", false);
}

private:
    void imprimir_(const NodoBST* n, std::string prefix,
        bool isLeft) const {
        if (!n) return;
        std::string conector = isLeft ? "+-- " : "-- ";
        std::cout << prefix << conector
            << n->dato.codigo << " [PPA:"
            << std::fixed << std::setprecision(1)
            << n->dato.ppa << "]\n";
        std::string ext = isLeft ? "|   " : "   ";
        imprimir_(n->izquierdo.get(), prefix + ext, true);
        imprimir_(n->derecho.get(), prefix + ext, false);
    }
}; // fin clase ArbolAcademico

} // namespace una_puno

```

ACTIVIDAD 12: C++ — Programa principal: datos de prueba y verificación

Objetivo: Escribir el main() con los mismos 7 registros de la Actividad 2, ejecutar todas las operaciones y verificar que los resultados coincidan con Python.

El main() es la función de entrada del programa en C++. Equivale al bloque `if __name__ == '__main__':` de Python. Use los mismos datos para poder comparar salidas directamente:

```

// — main.cpp —————
#include "ArbolAcademico.hpp"
#include <iostream>
#include <iomanip>

using namespace una_puno;

int main() {
    std::cout << "=== BST Sistema Academico UNA-PUNO (C++17) ===\n\n";

    ArbolAcademico arbol;

```

```

// Mismos 7 registros que Python (Actividad 2)
arbol.insertar({20210500,"Mamani Quispe, Juan", "Ing.
Sistemas",15.8f,120,EstadoAcademico::ACTIVO, "2021-I"});
arbol.insertar({20210300,"Huanca Apaza, Maria", "Ing. Civil",
14.2f,110,EstadoAcademico::ACTIVO, "2021-I"});
arbol.insertar({20210700,"Condori Flores, Pedro", "Medicina",
17.1f,130,EstadoAcademico::ACTIVO, "2021-I"});
arbol.insertar({20210100,"Ticona Lupaca, Rosa", "Contabilidad", 12.0f,
90,EstadoAcademico::SUSPENDIDO,"2021-I"});
arbol.insertar({20210400,"Larico Ccama, Carlos", "Ing.
Sistemas",16.5f,115,EstadoAcademico::ACTIVO, "2021-I"});
arbol.insertar({20210600,"Cutipa Vargas, Elena", "Agronomia",
13.7f,100,EstadoAcademico::ACTIVO, "2021-I"});
arbol.insertar({20210900,"Pari Choque, Luis", "Ing.
Sistemas",18.3f,140,EstadoAcademico::EGRESADO, "2021-I"});

// Estructura del árbol ASCII
arbol.imprimirArbol();

// In-Order – debe coincidir exactamente con Python
std::cout << "\n-- In-Order (ordenado por codigo) --\n";
std::cout << std::left
    << std::setw(12) << "CODIGO"
    << std::setw(35) << "NOMBRE"
    << std::setw(20) << "ESCUELA"
    << "PPA" << '\n';
std::cout << std::string(70, '-') << '\n';
for (const auto& e : arbol.inOrder()) e.print();

// Búsqueda
std::cout << "\n-- Busqueda --\n";
auto res = arbol.buscar(20210700);
if (res) {
    std::cout << "Encontrado: ";
    res->print();
}
auto noExiste = arbol.buscar(99999999);
std::cout << "Buscar 99999999: "
    << (noExiste ? "encontrado" : "no encontrado") << '\n';

// Consulta por rango PPA
std::cout << "\n-- Estudiantes con PPA >= 15.0 --\n";
for (const auto& e : arbol.porRangoPPA(15.0f)) e.print();

// Estadísticas
std::cout << "\n-- Estadísticas --\n";
arbol.estadisticas();

// Eliminar nodo con dos hijos
std::cout << "\n-- Eliminando codigo 20210300 --\n";
arbol.eliminar(20210300);
std::cout << "Nodos restantes: " << arbol.inOrder().size() << '\n';
std::cout << "In-Order: ";
for (const auto& e : arbol.inOrder())
    std::cout << e.codigo << " ";
std::cout << '\n';

// BFS
std::cout << "\n-- BFS (nivel por nivel) --\n";
for (const auto& e : arbol.bfs())
    std::cout << e.codigo << " ";
std::cout << '\n';

return 0;

```

}

Complete la siguiente tabla comparando las salidas de Python (Actividad 2–4) y C++ (Actividades 9–12). Ambas deben coincidir exactamente:

Verificación	Esperado	Resultado Python	Resultado C++	¿Coinciden ?
In-order: primer código	20210100			Sí / No
In-order: último código	20210900			Sí / No
Altura del árbol	2			Sí / No
Total nodos	7			Sí / No
Buscar 20210700 PPA	17.1			Sí / No
Buscar 99999999	no encontrado			Sí / No
Nodos tras eliminar 20210300	6			Sí / No

ACTIVIDAD 13: C++ — Benchmark y comparación Python vs C++

Objetivo: Medir tiempos de inserción y búsqueda en C++ con 100, 1000, 10000 y 100000 registros y comparar con los resultados de Python de la Actividad 6.

Use `std::chrono::high_resolution_clock` para medir tiempos con precisión de nanosegundos, y datos generados con `std::mt19937` (Mersenne Twister) para asegurar códigos únicos aleatorios:

```
// — benchmark.cpp —
#include "ArbolAcademico.hpp"
#include <chrono>
#include <random>
#include <algorithm>

using namespace una_puno;
using Clock = std::chrono::high_resolution_clock;

// Genera n estudiantes con códigos únicos aleatorios
std::vector<Estudiante> generarDatos(int n) {
    std::mt19937 rng(42); // semilla fija para reproducibilidad
    std::uniform_int_distribution<int> distCod(20000000, 29999999);
    std::uniform_real_distribution<float> distPPA(8.0f, 20.0f);

    std::vector<int> codigos;
    while ((int)codigos.size() < n) {
        int c = distCod(rng);
        codigos.push_back(c);
    }
    // Eliminar duplicados
    std::sort(codigos.begin(), codigos.end());
    codigos.erase(std::unique(codigos.begin(), codigos.end()), codigos.end());
    codigos.resize(n);
    // Mezclar para evitar árbol degenerado
    std::shuffle(codigos.begin(), codigos.end(), rng);
}
```

```

std::vector<Estudiante> datos;
for (int i = 0; i < n; i++)
    datos.emplace_back(codigos[i], "Estudiante_" + std::to_string(i),
                      "Ingenieria", 10.0f + distPPA(rng) * 0.5f,
                      100, EstadoAcademico::ACTIVO, "2024-I");

return datos;
}

void benchmark(int n) {
    auto datos = generarDatos(n);
    int buscarCod = datos[n / 2].codigo;

    // Inserción
    ArbolAcademico arbol;
    auto t0 = Clock::now();
    for (auto& e : datos) arbol.insertar(e);
    auto t1 = Clock::now();
    double ms_ins = std::chrono::duration<double, std::milli>(t1-t0).count();

    // Búsqueda
    auto t2 = Clock::now();
    arbol.buscar(buscarCod);
    auto t3 = Clock::now();
    double ms_bus = std::chrono::duration<double, std::milli>(t3-t2).count();

    std::cout << std::setw(8) << n
              << " | ins: " << std::fixed << std::setprecision(3)
              << std::setw(9) << ms_ins << " ms"
              << " | bus: " << std::setw(8) << ms_bus << " ms"
              << " | altura: " << arbol.altura() << '\n';
}

int main() {
    std::cout << "          N | Insercion          | Busqueda          | Altura\n";
    std::cout << std::string(60, '-') << '\n';
    for (int n : {100, 1000, 10000, 100000})
        benchmark(n);
    return 0;
}

```

Complete la siguiente tabla con los resultados de C++ y compárelos con los de Python (Actividad 6). Analice el factor de diferencia:

N	Ins. C++ (ms)	Ins. Py (ms)	Bus. C++ (ms)	Bus. Py (ms)	Altura	Factor
100						
1 000						
10 000						
100 000						

Factor = Tiempo Python / Tiempo C++. Un factor de 20 significa que C++ es 20 veces más rápido. Anote sus conclusiones sobre cuándo conviene usar cada lenguaje:

Conclusiones:

VI. PREGUNTAS DE REFLEXIÓN

- ¿Por qué el recorrido in-order de un BST produce siempre una lista ordenada ascendentemente? Explique usando la definición de la invariante de orden.
- ¿Qué ocurre con la complejidad del BST cuando los datos se insertan en orden creciente? ¿Cómo se llama esta situación y qué estructura la soluciona?
- Compare la búsqueda en un BST, en una lista Python y en un diccionario Python. ¿En qué situaciones reales preferiría usar un BST en lugar de un diccionario, sabiendo que el diccionario tiene $O(1)$ amortizado?
- El Linux Kernel usa Red-Black Trees en lugar de BST simples para gestionar los procesos del scheduler CFS. ¿Por qué no usa un BST simple? ¿Qué garantía adicional ofrece el Red-Black Tree?
- ¿Cómo se podría aplicar un BST (o un KD-Tree, su extensión espacial) para indexar los centros poblados de los 109 distritos de la Región Puno por coordenadas GPS, de modo que las consultas de cercanía sean $O(\log n)$ en lugar de $O(n)$?

VII. TRABAJO DE INVESTIGACIÓN

Tema: Árboles Binarios de Búsqueda y sus variantes en sistemas reales.

- Investigar dos sistemas de software reales (distintos a los mencionados en esta práctica) que usen BST o alguna variante (AVL, Red-Black Tree, B-Tree, B+-Tree, KD-Tree) como estructura de datos interna.
- Para cada sistema, describir: (a) el problema que resuelve, (b) por qué se eligió esa variante de árbol, (c) qué complejidad garantiza, y (d) qué impacto tiene en el rendimiento del sistema.
- Incluir al menos un diagrama que muestre cómo se estructuran los datos en el árbol para ese sistema específico.
- Extensión mínima: 2 páginas, con figuras propias, referencias APA 7.^a edición con DOI.

VIII. RÚBRICA DE EVALUACIÓN

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Insuf. (1)	Peso
Ejecución del código Python	Sin errores, todas las salidas son correctas y coinciden con lo esperado.	Errores menores, la mayoría de salidas son correctas.	Ejecuta parcialmente, algunos métodos fallan.	No ejecuta o no entrega código.	15%
Ejecución del código C++	Compila sin warnings, outputs idénticos a Python,	Compila con warnings, resultados mayormente correctos.	Compila pero falla en alguna operación.	No compila o no entrega.	15%

	benchmark completo.				
Comprensión del BST	Explica correctamente invariante, recorridos y los 3 casos de eliminación.	Comprende inserción y búsqueda; dificultades con la eliminación.	Comprensión superficial; confunde recorridos.	No demuestra comprensión.	20%
Análisis de complejidad	Analiza correctamente mejor, promedio y peor caso con justificación.	Identifica $O(\log n)$ y $O(n)$ pero sin justificación del peor caso.	Solo menciona Big-O sin análisis.	No analiza complejidad.	15%
Comparación Python vs C++ (Act. 13)	Tabla completa, factor calculado, conclusiones sustentadas con evidencia.	Tabla completa sin análisis del factor.	Tabla incompleta.	No compara.	15%
Trabajo de investigación	Dos sistemas reales, diagramas propios, análisis profundo, referencias DOI.	Dos sistemas, referencias básicas sin análisis profundo.	Un sistema, sin diagrama o referencias incompletas.	No entrega o copia sin citar.	10%

IX. REFERENCIAS

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.^a ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/>
- Knuth, D. E. (2011). The art of computer programming, Vol. 3: Sorting and searching (2.^a ed.). Addison-Wesley Professional.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4.^a ed.). Addison-Wesley Professional. <https://algs4.cs.princeton.edu/home/>
- Dasgupta, S., Papadimitriou, C., & Vazirani, U. (2006). Algorithms. McGraw-Hill Education. <https://people.eecs.berkeley.edu/~vazirani/algorithms/all.pdf>
- Bayer, R., & McCreight, E. M. (1972). Organization and maintenance of large ordered indexes. Acta Informatica, 1(3), 173–189. <https://doi.org/10.1007/BF00288683>
- Guibas, L. J., & Sedgewick, R. (1978). A dichromatic framework for balanced trees. Proceedings of the 19th Annual Symposium on Foundations of Computer Science (pp. 8–21). IEEE. <https://doi.org/10.1109/SFCS.1978.3>
- Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. Communications of the ACM, 18(9), 509–517. <https://doi.org/10.1145/361002.361007>
- Love, R. (2010). Linux kernel development (3.^a ed.). Addison-Wesley Professional. [Cap. 4: Process scheduling — Red-Black Trees in CFS]
- Python Software Foundation. (2024). collections.deque — Python 3.11 documentation. <https://docs.python.org/3.11/library/collections.html#collections.deque>
- MIT OpenCourseWare. (2011). 6.006 Introduction to algorithms — Lecture 5: Binary search trees. MIT. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/>